DSA — MODULE 3

Colourful, exam-focused master notes • Linear & non-linear data structures

Prepared from the uploaded course materials on Unit III Linear Data Structures, linked lists, tree data structures and hashing. Sorting algorithms are intentionally kept in a separate PDF.

> Exam-answer format: definition → labelled diagram or example → operations/algorithm → advantages and limitations → time complexity (where the source supports it).

1. Introduction to Data Structures

A data structure is a way of organizing and storing data in a computer so that it can be accessed and manipulated efficiently. It represents the logical organization of data and the relationships between data elements.

Classification	Examples	Core idea
Primitive	int, char, float, double, Boolean	Basic built-in data types.
Non-primitive — linear	Array, linked list, stack, queue	Elements are organized sequentially.
Non-primitive — non-linear	Tree, graph	Elements may have hierarchical or network relationships.
Static / fixed-size style	Array (as presented in the course material)	Capacity is determined in advance.
Dynamic	Linked list; linked implementations of stacks/queues	Structure can grow or shrink during execution.

Common operations

Traversal (visit elements), searching (find an item), insertion (add an item), deletion (remove an item), updating, and merging/concatenation are common data-structure operations.

> Choose a structure based on the operations that matter most: random access, frequent insertions/deletions, ordered processing, hierarchical data, or fast lookups.

2. Arrays and Lists

Array

An array stores elements of the same type in consecutive memory locations. An index identifies each element, so direct indexed access is efficient.

Operation	Typical cost	Explanation
Access A[i]	O(1)	The index identifies the location directly.
Search (unsorted)	O(n)	May need to inspect every element.
Insert/delete near beginning or middle	O(n)	Elements may need to be shifted.
Insert at end when free capacity exists	O(1)	No shifting is required in that case.

Advantages: direct access by index and simple implementation. Limitations: fixed capacity in a basic static array, possible unused capacity, and expensive insertion/deletion away from the end.

List ADT

A List ADT describes a sequence of zero or more elements and the operations available on it, independently of how the list is implemented. The uploaded material lists creating, traversing, inserting, deleting and concatenating lists as basic operations.

Array list vs linked list

Feature	Array-based list	Linked list
Memory	Contiguous element storage	Nodes may be anywhere in memory; connected with pointers.
Indexed access	O(1)	O(n) traversal in a singly linked list.
Insert/delete in middle	Usually O(n) due to shifts	Can be O(1) after the target position/node is reached.
Size	Capacity management needed	Can grow/shrink as nodes are added/removed.
Best suited for	Frequent random access	Frequent structural updates and sequential access.

Important wording: linked-list insertion is not automatically O(1) if you must first traverse to find the insertion position. The pointer update can be O(1) once the position is known.

3. Linked Lists

A linked list is a dynamic data structure made of nodes. A node contains data and one or more links/pointers. In a singly linked list, a header/head pointer identifies the first node and the final node points to NULL.

Diagram: HEAD → [data | next] → [data | next] → [data | NULL]

Singly linked list: basic C++ node

```
struct Node {
    int data;
    Node* next;
    Node(int value) : data(value), next(nullptr) {}
};
```

Traversal

```
Node* current = head;
while (current != nullptr) {
    cout << current->data << " ";
    current = current->next;
}
```

Insertion at the beginning

```
Node* newNode = new Node(value);
newNode->next = head;
head = newNode;
```

Deletion at the beginning

```
if (head != nullptr) {
    Node* temp = head;
    head = head->next;
    delete temp;
}
```

Operation	Singly linked list (typical)	Reason
Access/search by value	O(n)	May traverse from head.
Insert at head	O(1)	Change two pointers.
Delete at head	O(1)	Advance head pointer.
Insert after a known node	O(1)	Update links.
Insert at tail without tail pointer	O(n)	Traverse to the last node.

Types of linked lists

Type	Node links / ending	Use or feature
Singly linked	Next pointer; last points to NULL	Forward traversal.
Doubly linked	Previous and next pointers	Forward and backward traversal; deletion can be easier when node is known.
Circular linked	Last node links back to first node	Repeated cyclic traversal; no NULL at the circular link.

Advantages and limitations

Advantages: dynamic size, no need for contiguous memory, and convenient link changes for insertion/deletion. Limitations: no direct indexed access, pointer storage overhead, and traversal is sequential.

> For a 5- or 8-mark answer, draw a node with its data and pointer fields, explain HEAD and NULL, then describe traversal, insertion and deletion.

4. Stacks

A stack is a linear data structure in which insertion and deletion occur at one end called TOP. It follows LIFO — Last In, First Out.

Example: push 10, push 20, push 30 → top is 30; the first pop returns 30.

Operation	Meaning	Typical array/linked stack cost
push(x)	Insert x at TOP	O(1)
pop()	Remove and return TOP item	O(1)
peek()/top()	Read TOP without removing	O(1)
isEmpty()	Check whether stack has no items	O(1)

Applications from the course concepts

Program control, recursion/call management, expression processing, undo operations and backtracking are common stack applications.

> Array stack warning: overflow occurs when a fixed-capacity stack is full; underflow occurs when pop is attempted on an empty stack.

5. Queues

A queue is a linear data structure where insertion takes place at the REAR and deletion at the FRONT. It follows FIFO — First In, First Out.

Example: enqueue A, B, C → dequeue returns A first.

Operation	Meaning	Typical cost
enqueue(x)	Insert x at rear	O(1) with a suitable implementation
dequeue()	Remove item at front	O(1) with linked or circular-array implementation
front()/peek()	Inspect first item	O(1)
isEmpty()	Check whether queue is empty	O(1)

Queue variants

A simple queue uses FRONT and REAR. A circular queue reuses array positions by wrapping around. A priority queue serves items according to priority rather than strictly by arrival order (implementation determines costs).

> Stack = LIFO, one active end (TOP). Queue = FIFO, insertion at REAR and deletion at FRONT.

6. Trees

A tree is a non-linear structure whose nodes are connected by edges. It models hierarchical relationships. The uploaded Tree Data Structure material explains a root, parent/child relationships, subtrees, leaves, siblings, depth and height.

Tree terminology

Term	Meaning
Root	Distinguished top node; has no parent.
Parent / child	A parent has a direct downward connection to its child.
Leaf / external node	Node with no children.
Internal node	Node with at least one child.
Siblings	Nodes sharing the same parent.
Depth of a node	Number of edges from root to that node.
Height of a node	Number of edges on the longest downward path from that node to a leaf.
Height of a tree	Height of its root.

General tree and binary tree

A general tree node may have any number of children. A binary tree is a specialized tree in which each node has at most two children, commonly called left and right.

Type	Definition
Full binary tree	Every node has either zero or two children; no node has exactly one child.
Complete binary tree	All levels are full except possibly the bottom level, which is filled from left to right.
Binary Search Tree (BST)	A binary tree ordered so that values in the left subtree precede the node and values in the right subtree follow it, under the usual distinct-key convention.

Tree traversals

Traversal	Visit order	Memory cue
Preorder	Root → Left → Right	Root first.
Inorder	Left → Root → Right	Root in the middle; for a BST, produces sorted order under standard ordering.
Postorder	Left → Right → Root	Root last.
Level order	Level by level, usually using a queue	Breadth-first.

BST search

At each node, compare the target with the node key. If equal, found. If smaller, move left; if larger, move right. A balanced BST can search in O(log n); an unbalanced BST may take O(n) in the worst case.

> The uploaded tree handout illustrates the BST property through inorder traversal. Its page 4 example visits left child, root, then right child; a balanced BST can be searched in O(log n).

7. Graphs — basic introduction

A graph consists of vertices (nodes) and edges connecting them. Edges may be directed or undirected and may carry weights. A weight can represent cost, distance or another quantity associated with traversing an edge.

Term	Meaning
Vertex	A node in the graph.
Edge	Connection between two vertices.
Directed graph	Edges have a direction.
Undirected graph	Edges have no direction.

Term	Meaning
Weighted graph	Edges carry numerical weights.
Path	Sequence of vertices connected through edges.

Applications include route networks, relationships, dependency modelling and computing path costs. The Unit III source introduces graphs as linked structures containing vertices and edges, with weights used to compute path-traversal cost.

8. Hashing

Hashing maps a key to an index in a hash table using a hash function. It is useful when frequent lookups and updates are needed; the source motivates hashing as a way to aim for near-constant-time O(1) operations.

Hash function

A hash function h(key) computes a table position. A good function should be easy to compute and distribute keys evenly. The hashing lecture notes suggest that a prime table size can help distribution.

```
h(key) = key mod table_size

Example: table size = 7
h(24) = 24 mod 7 = 3
So key 24 maps to index 3.
```

Collision

A collision occurs when two different keys map to the same table index. Collisions are expected because many possible keys may map into a smaller table.

Collision-resolution techniques

Technique	Idea	Important note
Separate chaining	Each table slot stores a list of keys that map to that slot.	Load factor λ = number of keys / table size; λ may exceed 1.
Linear probing	If the target slot is occupied, check subsequent slots.	A common formula is h(key,i) = (h(key)+i) mod N.
Quadratic probing	Probe using quadratic offsets.	Can reduce primary clustering compared with linear probing; details depend on the chosen formula.
Double hashing	Use a second hash function to determine the probe step.	Step size should be chosen carefully to visit available slots.

Linear probing worked example

Use table size N = 6 and h(key) = key mod 6. Insert 11, 10, 17, 16, 23 in this order.

Key	Home index	Placement with linear probing
11	5	Index 5
10	4	Index 4
17	5	5 occupied → index 0
16	4	4 occupied → 5 occupied → index 0 occupied → index 1
23	5	5, 0, 1 occupied → index 2

Hash-table operations are O(1) on average with a good hash function and controlled load factor; worst-case search can be O(n), for example when many keys collide.

9. High-yield comparison tables

Structure	Access pattern	Insert/delete strength	Typical search
Array	Direct indexed access	At end can be cheap; middle/front shifts cost O(n)	O(n) unsorted search
Linked list	Sequential traversal	Pointer updates can be cheap after reaching position	O(n)
Stack	TOP only	Push/pop at TOP: O(1)	Search O(n)
Queue	FRONT/REAR	Enqueue/dequeue can be O(1)	Search O(n)
Balanced BST	Tree-guided	Depends on balance and operation	O(log n)
Hash table	Hash-indexed	Average O(1) insert/update	Average O(1), worst O(n)

10. Exam-ready questions

1. Define a data structure and classify linear and non-linear data structures with examples.
2. Compare arrays and linked lists. Explain why insertion/deletion in an array may require shifting.
3. Draw a singly linked list and write C++ code for traversal and insertion at the beginning.
4. Differentiate singly, doubly and circular linked lists.
5. Define stack and queue. Compare LIFO and FIFO with operations and applications.
6. Explain tree terminology: root, leaf, internal node, sibling, depth and height.
7. Differentiate full and complete binary trees. Explain BST search and inorder traversal.
8. Define hashing and collision. Explain separate chaining and linear probing with an example.
9. Explain load factor and discuss why a good hash function is important.
10. Compare arrays, linked lists, balanced BSTs and hash tables for searching.

11. One-page last-minute revision

Remember	Exam trigger
Array	Contiguous memory; index-based random access.
Linked list	Nodes + pointers; dynamic size; sequential access.
Stack	LIFO; TOP; push/pop.
Queue	FIFO; insert REAR; delete FRONT.
Tree	Root, parent/child, leaves, depth/height; non-linear.
BST	Search left or right based on key comparison; balanced search O(log n).
Graph	Vertices + edges; may be directed/weighted.
Hashing	Key → table index; collisions need handling.
Separate chaining	List of keys at a slot.
Linear probing	Try next available index; clustering may occur.

Source alignment note: These notes summarize and organize the uploaded material into revision form. Where a detail is not explicitly covered in the supplied pages, it is framed as a standard supporting explanation rather than a direct quotation.